==
ANTDESIGNER - FULL REBUILD: STREAMLIT -> HANDCRAFTED HTML/CSS/JS + PYTHON API
Prompt for the coding agent (Arena). Read ALL of it before touching anything.
==

0. WHO / WHAT / WHY

I am Kara. The project is "AntDesigner" (repo: kara-nawzad/Antenna-AI-Project, live at antenna-ai-project.fly.dev): a graduation project from Sulaymaniyah Polytechnic University that predicts S11/VSWR/gain/bandwidth of CPW microstrip patch antennas (12 shapes, 4 families) and does inverse design (give a target frequency, get the dimensions). It is currently a ~2000-line Streamlit app (10_web_app.py) plus engine_v2.py, antenna_viz.py, antenna_charts.py.

Streamlit cannot give me the interface I designed. Two attempts to restyle it failed (stripped scripts, CSS fighting Streamlit, no hero, default tabs, no motion). The design is finished and approved. It exists as a working HTML mockup: reference_preview.html (attached). The job now:

   KEEP the Python science exactly as it is.
   REPLACE the Streamlit front end with a real website built from
   HTML + CSS + JavaScript, talking to a small Python API (FastAPI).

The result must look and move like reference_preview.html - not "inspired by" it - and must work with the REAL engine, the REAL data, the REAL chatbot.

You have freedom on implementation details (section 4 says where). You have no freedom on the non-negotiables (section 2).

1. WHAT YOU RECEIVE

- This prompt.
- reference_preview.html : the approved design, fully working with DEMO data. OPEN IT IN A BROWSER FIRST. Take screenshots of every tab at 1440px and 390px width and keep them: they are your visual target and your diff baseline. Read its source: it contains the exact CSS tokens, markup, the hero SVG (drifting ridgelines + antenna mast), the tab bar with sliding underline, scroll-reveal, count-up and the animated antenna / S11 charts.
- antenna_viz.py and antenna_charts.py : light-palette versions of the repo's modules. They add antenna_animated_html(...) and s11_vswr_animated_html(...) which return the self-drawing animated components as HTML strings. Geometry code is unchanged. Use them.
- assets/ : AntDesigner logo/icon/favicon, SPU logos (original colours).

- antdesigner_theme.py : OPTIONAL reference only (it is the Streamlit version of the theme; its render_hero / render_team / ridge-SVG code is a handy copy-source for markup and wording). Do not ship it.
- The real repo (link or zip). The Hugging Face model hub is kara-nawzad/antenna-models (models/CSV are downloaded at first boot).

2. NON-NEGOTIABLES

2.1 engine_v2.py: DO NOT MODIFY. Not one character.

2.2 Model files, CSV data, scalers, Hugging Face repo: DO NOT MODIFY, retrain, re-export, rename, or re-upload.

2.3 NUMERICAL IDENTITY. The new backend must return exactly the same numbers the Streamlit app returns today for the same inputs (tolerance 1e-9). You will prove it (section 7). The scientific functions are copied, not rewritten, not "cleaned up", not "optimised".

2.4 Work on a NEW BRANCH: `html-rebuild`. Leave 10_web_app.py and the Streamlit setup untouched on main so I can roll back by redeploying main. (You may keep 10_web_app.py in the branch unchanged; do not delete it.)

2.5 Do not push to main, do not merge, do not deploy to fly.io, do not touch Hugging Face, never print or commit secrets (GROQ_API_KEY).

2.6 NO FABRICATED DATA. The demo numbers inside reference_preview.html must never appear in the real site. Every number on screen comes from the API.

2.7 FEATURE PARITY. Every feature, control, caption and text block that exists in 10_web_app.py today exists in the new site (section 5 lists them; you must also verify by reading the file). Project wording (intro, system overview, limitations, methodology, captions) is copied VERBATIM from the Streamlit app. The only new copy is: hero, "How it works", team block (all given in the reference), plus small UI states (loading/error).

2.8 Team credits are exactly: Prepared by Arwin Hunar, Dunya Anwar, Kara Nawzad, Khazan Jafaar, Ramyar Aram, Ranw Jamil. Supervised by Dr. Halgurd N. Awl. Sulaymaniyah Polytechnic University.

2.9 Deployment limits: the production machine is a small fly.io shared-CPU VM (about 1 GB RAM), TensorFlow is the CPU build. Keep memory use at or below what Streamlit uses today. No new heavy dependencies.

2.10 Dependencies: only add what is required (fastapi, uvicorn, and plotly if missing from the image - antenna_charts imports it). Pin versions you add. Fix the known Groq bug (section 6.7).

2.11 If anything is ambiguous or contradicts this file: STOP and ask me. Do not guess on anything touching the engine or the numbers.

3. WHAT "DONE" LOOKS LIKE (the acceptance test)

I open the site and see, without me explaining anything:

- A header with the AntDesigner logo and the five tabs. A sliding underline moves under the active tab. Tabs are pinned at the top and work on phones

(horizontal scroll). URL hash routing: #home #forward #inverse #aether #method; back/forward buttons work; refresh keeps the tab.
- Home: the hero. Headline words rise one by one ("We taught a neural network to design antennas."), slowly drifting mountain ridgelines in the background with a small antenna mast emitting signal arcs, light parallax on pointer move. Then How it works (3 steps), Project Introduction, System Overview, the 12 shapes / 4 families, Scope and limitations, team block with the SPU logo in its original colours, footer.
- Paper background (#F5F2EA), navy ink, ONE purple accent used for actions and data. Fraunces headlines (one italic purple word), Inter text, JetBrains Mono numbers. Hairline borders, 10-14 px radius, no glow, no gradients, no glass, no dark mode.
- Forward tab: pick shape, Lp, Wp (slider and number box stay in sync, clamp 5-40 mm), run, and get real results: animated metrics counting up, the self-drawing S11/VSWR chart with hover crosshair, the animated antenna drawing (2D) and the 3D view, comparison overlay, design-space map, report download.
- Inverse tab: target frequency -> real optimized geometry + the visuals the Streamlit version shows.
- AETHER tab: a real working chat with the Groq assistant.
- Method & Validation tab: the real self-tests, A/B test, inverse test and health check, with their real output.
- Everything reveals on scroll, nothing flashes unstyled, nothing jumps. It is fast, and it works at 390 px.

If any of that is missing, the work is not done, whatever the code says.

4. ARCHITECTURE (recommended; you may choose better, but justify in writing)

One container, one URL, one fly app (same as today):

```
repo/
  backend/
    main.py            FastAPI app: API routes + serves the frontend
    engine_core.py     science code moved out of 10_web_app.py (see 5.1)
    viz.py             (optional) thin wrappers around antenna_viz/charts
  engine_v2.py         UNCHANGED
  antenna_viz.py       the light-palette version supplied
  antenna_charts.py    the light-palette version supplied
  frontend/
    index.html
    css/app.css
    js/app.js          (split into modules if it helps; no build step
                        unless you can justify it - plain files preferred)
    assets/            logos, favicon
    fonts/             self-hosted Fraunces, Inter, JetBrains Mono (woff2)
  tests/
```

```
      test_parity.py     proves numerical identity (section 7)
    10_web_app.py        LEFT UNTOUCHED (legacy)
  Dockerfile, fly.toml, requirements.txt : minimal edits only (section 8)
```

Why plain HTML/CSS/JS and not React: the design is a handcrafted page with custom animation; a build pipeline adds risk and weight for no benefit. Use React/Vite ONLY if you can show it is clearly safer, and then keep the output static and served by FastAPI.

Fonts: self-host woff2 files so the first paint never depends on Google. If you have no network, load them from Google Fonts with real fallback stacks (serif / system-ui / ui-monospace) and tell me.
Libraries: only from a pinned version on cdnjs/jsdelivr or self-hosted (Plotly.js for 3D / comparison / design-space charts is acceptable, pinned). No heavy UI frameworks.

5. THE BACKEND

5.1 EXTRACTION (the delicate step)

Today the science lives inside 10_web_app.py, mixed with Streamlit. Move it, unchanged, into backend/engine_core.py:

constants:	FREQS_RAW, RESONANCE_ALIGN, ML_BLEND_WEIGHT, ML_BLEND_RADIUS_MM, TRAINED_MIN_MM, TRAINED_MAX_MM, SHAPES, FAMILY_MAP, KNOWN_META_COLS, HF_REPO
loading:	_download_hf, _load_scaler, load_brains, load_data
helpers:	out_of_training_range, clamp_5_40, apply_dsp_polishing, vna_measure, curve_columns, align_curves, weighted_blend_from_rows, grid_bilinear_estimate, local_plane_estimate
forward:	forward_ml, forward_anchor, forward_best, forward_physics_first, forward_physics_first_v2, nearest_csv_row
inverse:	run_inverse_solver
validation:	run_engine_self_test, run_engine_ab_test, run_inverse_self_test, run_startup_healthcheck

Rules for the move:
- Copy the function bodies verbatim. The ONLY edits allowed are removing Streamlit: replace @st.cache_resource / @st.cache_data with a thread-safe lazy singleton (functools.lru_cache or a module-level loader with a lock), and delete st.* calls (spinners, session_state, widgets). The sync_lp_*/ sync_wp_* callbacks are UI only; they become frontend code.
- Keep `import engine_v2` guarded exactly as today (HAS_V2 fallback).
- Keep the exact same call used by the Forward button today: forward_physics_first_v2(lp, wp, shape_id, v2_fwd, v2_dip_specialist,

s_geo, s_perf, db, use_ml_refine=..., blend_window_mhz=30, anchor_strength=0.3) when v2 is available, else forward_physics_first(...). Then nearest_csv_row for the "nearest CST anchor" caption.
- Load models ONCE, in a background thread at startup, so the server answers /api/health immediately while TensorFlow loads (fly health checks and the UI both need this). The UI shows an "engine is warming up" state until ready (poll /api/health).
- Leave a comment at the top of engine_core.py: which lines of 10_web_app.py each function came from.

5.2 API (JSON; adjust names if you have a better scheme, keep the content)

```
GET  /api/health      -> {status, engine_ready, v2_loaded, rows, versions}
GET  /api/meta        -> shapes, families, priorities, slider range 5-40,
                         trained range 10-35, ML_BLEND_WEIGHT,
                         ML_BLEND_RADIUS_MM, TRAINED_* (so the UI never
                         hard-codes science numbers)
POST /api/forward     {shape_id, lp, wp, use_ml_refine}
     -> everything the Streamlit result dict has: status, engine, fr, bw,
        s11, gain, eff, f[], c[], v[], vswr_at_min, near_row{lp,wp,fr,s11,
        dist}, out_of_range (bool), neural_note: one of "active" |
        "not_applied_near_cst" | "not_applied" | "off" (same logic as the
        Streamlit messages)
POST /api/visual/antenna  {shape_id, lp, wp} -> {html}  (from
     antenna_viz.antenna_animated_html; shown in a sandboxed iframe srcdoc)
POST /api/visual/response {f,c,v,fr,bw,title} -> {html}  (from
     antenna_charts.s11_vswr_animated_html)
POST /api/visual/3d       {shape_id, lp, wp} -> Plotly JSON
POST /api/visual/compare  {entries:[{label,f,c}]} -> Plotly JSON
GET  /api/visual/design-space?shape_id=.. -> Plotly JSON or null
     (visual endpoints may be folded into /api/forward if cleaner; decide
      for speed and simplicity, keep each response fast)
POST /api/inverse     {target_fr, family, priority}
     -> lp, wp, shape_id, fr, err_pct, gain, bw, status, priority
        (+ whatever extra the Streamlit inverse section shows: read
        10_web_app.py lines ~1895-1990 and mirror all of it)
POST /api/aether      {messages:[{role,content}]} -> {reply, model}
POST /api/validate/self-test | ab-test | inverse-test | healthcheck
     -> the same stats dicts the Streamlit version builds its tables and
        scatter plots from. These can take a long time: run them as
        background jobs (POST starts, GET polls with progress) or allow
        long timeouts; the UI must show progress and never freeze.
```

5.3 Hardening: validate inputs (shape 1-12, 5<=lp,wp<=40, 1<=target<=7), return clear JSON errors, never a stack trace to the browser. NaN/inf are converted to null. CORS not needed (same origin).

5.4 Serve frontend/ with StaticFiles at "/", API under "/api". Correct cache headers: long cache for fonts/assets, no-cache for index.html.

6. THE FRONTEND

6.1 Start from reference_preview.html. Split it into index.html, app.css, app.js (+ assets). Keep its markup, class names, tokens and motion. Replace demo data with fetch() calls to the API. Remove all demo numbers and any "demo" labels only where real data now exists.

6.2 Design tokens (use CSS variables, defined once):
paper #F5F2EA | card #FFFDF8 | ink #14213D | muted #5B6478 |
accent #6366F1 / #4F46E5 | hairline rgba(20,33,61,.14)
Fonts: Fraunces (display; one italic accent word per headline), Inter (UI), JetBrains Mono (numbers, inputs). Radius 10-14px. 8px spacing grid. Buttons are FLAT purple (no heavy saturated blue, no glow). Hover lifts 2px, press settles.

6.3 Motion (all required, all purposeful, all disabled under prefers-reduced-motion with content fully visible):
1 hero headline words rise in sequence; 2 ridgelines drift slowly + pointer parallax (desktop only); antenna mast arcs pulse; 3 sections reveal on scroll once; 4 sliding tab underline; 5 metrics count up when results arrive; 6 charts and antenna drawings draw themselves, chart hover crosshair; 7 cards lift on hover; 8 tab content cross-fades.
Never: bounce, spin, hide content behind animation, jank on scroll.

6.4 States (never leave a blank or raw error):
- engine warming up (poll /api/health; friendly message + subtle progress)
- running simulation: short, honest loading state
- error: a calm message in the card, with a retry
- empty: e.g. "Add a second design to overlay the curves."
- out-of-range warning (outside 10-35 mm): same wording as Streamlit, styled as a calm note, not a red alarm.
- AETHER with no key: friendly "assistant not configured" card.

6.5 Responsive (390 px is a first-class target): stacked layout, horizontal scrolling tab bar, 44px tap targets, no horizontal page scroll, charts resize. Check 390, 768, 1280, 1440.

6.6 Accessibility and quality: semantic HTML, labelled inputs, visible focus rings, keyboard-operable tabs (arrow keys), alt text, ::selection colour, real favicon, <title> and meta description, contrast >= 4.5:1.

6.7 AETHER chat: keep the model-fallback chain and "try the last working model first" logic exactly as in the Streamlit code (the list in 10_web_app.py). Conversation history lives in the browser; the server is stateless. Key comes from the GROQ_API_KEY environment variable ONLY. Messages render markdown safely (sanitize; no raw HTML injection).

KNOWN BUG TO FIX: the live site currently shows
TypeError: Client.__init__() got an unexpected keyword argument 'proxies'
This is the unpinned `groq` SDK meeting httpx >= 0.28. Fix by pinning a compatible pair in requirements.txt (e.g. httpx<0.28 or a newer groq release), then PROVE it: construct Groq(api_key="test") in the built image without error. Report which pins you chose.

6.8 Content map (read 10_web_app.py section by section; mirror everything):
HOME: hero, how it works, Project Introduction, System Overview, the 12 shapes / 4 families, Scope and limitations, team, footer.
FORWARD: Design Profile (12 shapes), Lp and Wp (slider + number input, synced, clamp 5-40), neural-refinement checkbox with its explanation text, "Initiate Simulation", Status heading, out-of-range warning, engine mode caption, neural-refinement note, 6 metrics (Resonance, Bandwidth, S11 Min, VSWR @ Min, Gain, Efficiency), nearest CSV anchor caption, S11 & VSWR chart, 2D layout / 3D structure tabs with the CPW caption, Design Toolkit (Add to comparison [keep last 6], Clear, compare chart when >=2, "Download design report" with the exact same report text format, generated client-side), "Explore the design space" panel with its captions.
INVERSE: Target Frequency (1-7, step .01, default 3.5), Optimization Priority (Balanced / Maximize Gain / Maximize Bandwidth), Shape Family, "Synthesize Optimal Geometry", result banner, Optimized Geometry metrics, and everything else that section shows today.
AETHER: the chat.
METHOD & VALIDATION: Engine Validation (held-out samples slider 40-400 step 20, Run Self-Test), A/B comparison, inverse self-test, startup health check, the methodology text, error scatter charts.
If the Streamlit app has something this list forgot, it still gets built.
Deliver a PARITY TABLE in the report (feature | old line | new location).

7. VERIFICATION (mandatory; report evidence, not claims)

7.1 tests/test_parity.py: for >= 40 inputs spread across all 12 shapes, including points near CST data, far from it, and outside 10-35 mm, and both use_ml_refine values: compare the NEW backend's outputs (fr, bw, s11, gain, eff, f, c, v, status, engine, near_row) with the ORIGINAL functions from 10_web_app.py. Max absolute difference must be <= 1e-9. Do the same for run_inverse_solver (all 4 families x 3 priorities x 3 target frequencies). Paste the summary table in your report. If you cannot import the old functions without Streamlit, run the old file's functions in a shim and say how.
7.2 Run the server locally. In a real browser (Playwright/Chromium) take screenshots at 1440 and 390 of: Home (top, middle, bottom), Forward (before and after a run, 2D and 3D, comparison with 2 designs, design

space open), Inverse (before and after), AETHER (with and without key), Method (after a self-test). Put them next to the reference screenshots and fix every visible difference.

7.3 Browser console: zero errors, zero warnings you caused. Network: no 404s, no blocked requests, no external request except the pinned CDNs you chose (ideally none).

7.4 Motion check: record or step through the hero, tab underline, scroll reveal, count-up, chart draw. Then test with reduced-motion on.

7.5 Click every control. Try bad input (empty, 0, 99, text). Try refresh on each #hash. Try back/forward. Try double-clicking Run. Try the engine still warming up.

7.6 `docker build` and `docker run -p 8080:8080` the image; hit /api/health; confirm the site works from inside the container; note the cold-start time, idle RAM and RAM after a run (must fit a ~1 GB machine).

7.7 If a step cannot be verified (no network, no Docker, no GPU, no Groq key), say exactly that. Never write "should work".

8. DEPLOYMENT FILES (the only infra edits allowed; list them in the report)

- Dockerfile: change the ENTRYPOINT/CMD from `streamlit run ...` to `uvicorn backend.main:app --host 0.0.0.0 --port 8080`; change HEALTHCHECK to curl http://localhost:8080/api/health (keep the long start period, models download on first boot); remove STREAMLIT_* env vars. Everything else stays.
- fly.toml: keep app name, region, port 8080, env and machine size. Adjust only what the new health path or request timeouts require (validation jobs can be slow). Do NOT deploy.
- requirements.txt: add fastapi, uvicorn[standard] (pinned); keep tensorflow-cpu etc.; keep or drop streamlit? Keep it ONLY if 10_web_app.py must still run on this branch; otherwise leave it so rollback stays easy.
- Provide the exact commands for me: run locally, build the image, deploy to fly (I will run the deploy myself), roll back to the Streamlit version.

9. WORKING METHOD

Step 0 RECON (no edits): read the whole repo and all of 10_web_app.py; list every st.* feature; open reference_preview.html and screenshot it; write down your architecture choice and any deviation from section 4.

Step 1 Branch `html-rebuild`. Extract engine_core.py + parity test FIRST. Nothing else starts until parity passes.

Step 2 API with the engine, tested with curl.

Step 3 Frontend from the reference, wired tab by tab: Home, Forward, Inverse, AETHER, Method.

Step 4 Polish: states, responsive, accessibility, motion, reduced-motion.

Step 5 Full verification (section 7), fix, re-verify.

Step 6 Deployment files (section 8), docker build/run.
Make small, well-named commits. After each step, run its check before moving on.

10. LESSONS FROM THE LAST FAILED ATTEMPTS (do not repeat)

- Scripts in st.markdown are stripped: motion silently disappeared. Here JS is a real file; make sure it actually runs.
- Two CSS systems fighting (old dark CSS + new theme). Here there is exactly ONE stylesheet.
- A hidden helper element leaked the text "pass" on the page. No stray debug text, no leftover Streamlit remnants.
- The hero, header and tabs were missing and the page opened on an empty gap.
- Default-looking widgets and a saturated blue button slipped through.
- The agent edited forbidden files (engine, fly.toml, Dockerfile) and called the job done without screenshots.

Do not claim done without the screenshots, the parity table and the console clean report.

11. FINAL REPORT (exactly this structure)

1. Architecture chosen + any deviation from this prompt and why.
2. File tree (new / changed / untouched) and `git diff --stat` vs main.
   Confirm: engine_v2.py, model files, CSV: byte-identical to main.
3. Parity table (feature | old location | new location | status).
4. Numerical parity results (section 7.1 summary).
5. Screenshots (1440 + 390) vs reference, with notes on any difference.
6. Console/network cleanliness, reduced-motion result, click-through result.
7. Docker: build ok, health ok, cold start, RAM.
8. Pinned versions added/changed (incl. the Groq/httpx fix).
9. What I could not verify, and known gaps.
10. Commands: run locally, build, deploy, roll back.